

von Komponenten mit Sicherheitslücken

ponenten wie Bibliotheken, Frameworks oder andere Soft-
en, kann es sein, dass diese externen Komponenten selbst Si-
alten, was wiederum Ihre Anwendung ebenfalls angreifbar

ponenten mit Sicherheitslücken

a, sollten Sie externe Komponenten, die Sie innerhalb einer
enden, immer auf dem aktuellsten Stand halten. Werden
er externen Komponente erkannt und behoben, sollten Sie
isieren.

s sinnvoll, auf automatisierte Tools zurückzugreifen, die Ihnen
hängigkeiten automatisch zu ermitteln. Insbesondere bei der
en Sie schnell dutzende oder hunderte (direkte oder indirekte)
em Projekt, sodass es äußerst aufwändig wäre, für all diese Ab-
nach Updates zu suchen. Tools wie »npm-check-updates«
([m/package/npm-check-updates](https://package/npm-check-updates)) oder der über npm verfügba-
([ps://docs.npmjs.com/cli/audit](https://docs.npmjs.com/cli/audit)) helfen Ihnen, Abhängigkeiten
es hin zu überprüfen.

e Protokollierung und Überwachung

dung unzureichend protokolliert oder überwacht wird und
oder Angriffe auf die Anwendung erst verspätet erkannt wer-
e Sicherheitslücke vor.

reichende Protokollierung und Überwachung

dass auffälliges Verhalten wie beispielsweise fehlgeschlagene
derholte Zugriffe sowie serverseitige Fehler protokolliert wer-
darauf achten, dass die Fehler mit einem ausreichenden Kon-
en, um auf diese Weise verdächtige Aktivitäten und die damit
a identifizieren. Außerdem sollten Sie die Protokolle bzw. Logs
en, das zur späteren Analyse in entsprechende Tools geladen
iel für ein solches Tool ist der sogenannte ELK-Stack (<https://monitoring>), bestehend aus Elastic Search, Logstash und Kaba-
eine grafische Oberfläche zur Log-Analyse bereitstellt.

17.1.12 Ausblick

Sie kennen jetzt die wichtigsten Sicherheitslücken von Webanwendungen und wis-
sen, wie Sie diesen durch entsprechende Maßnahmen vorbeugen können. Für einige
Maßnahmen allerdings – auf die ich bisher noch nicht im Detail eingegangen bin –
muss ich im Folgenden etwas weiter ausholen.

17.2 Verschlüsselung und Kryptografie

Damit Daten im Internet sicher übertragen werden können, müssen sie zunächst
verschlüsselt werden. Dazu existieren verschiedene Algorithmen (*Cryptographic Al-
gorithms*), die unter Verwendung eines Schlüssels (des *Encryption Key*) Daten **ver-
schlüsseln**, die anschließend nur unter Verwendung eines passenden Schlüssels (des
Decryption Key) **entschlüsselt** werden können.

Solche Arten von Algorithmen lassen sich wiederum in verschiedene Klassen unter-
teilen, die in Abbildung 17.3 zu sehen sind und auf die ich im Folgenden näher einge-
hen möchte: die *symmetrische Kryptografie* und die *asymmetrische Kryptografie*.

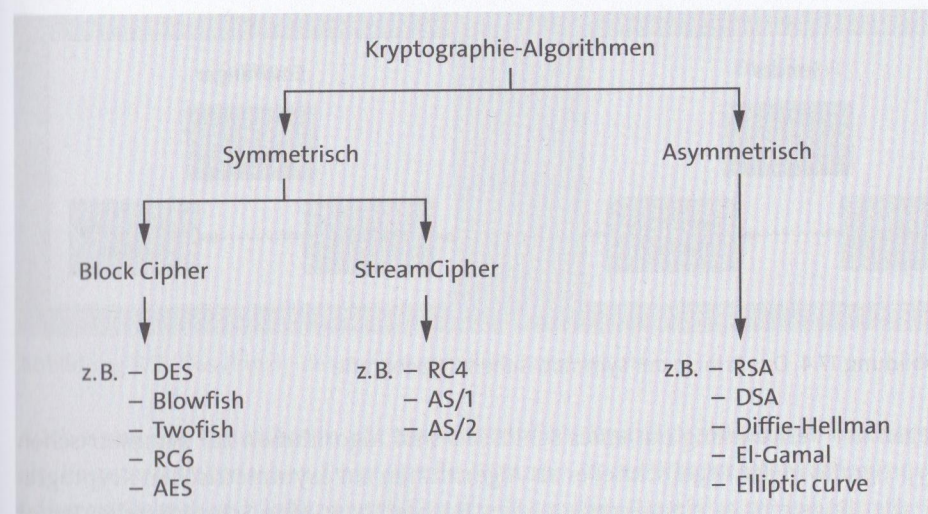


Abbildung 17.3 Klassifikation von Kryptografie-Algorithmen

Hinweis

Symmetrische Verschlüsselung wird weiter in *Blockverschlüsselung* (*Block Cipher*)
und – etwas holprig übersetzt – *Stromverschlüsselung* (*Stream Cipher*) unterteilt. Die
Details hierbei sind für uns an dieser Stelle allerdings weniger relevant.

Hinweis

Auch wenn in diesem Zusammenhang oft von Verschlüsselung gesprochen wird, ist eigentlich *Kryptografie* gemeint: Daten sollen ja nicht nur verschlüsselt, sondern auch wieder entschlüsselt werden.

17.2.1 Symmetrische Kryptografie

Bei der *symmetrischen Kryptografie* wird für die Verschlüsselung der Daten der **gleiche Schlüssel** verwendet wie für die Entschlüsselung der Daten. Der Sender von Daten, der die Daten verschlüsselt, verwendet also den gleichen Schlüssel wie der Empfänger der Daten, der die Daten entschlüsselt (Abbildung 17.4).

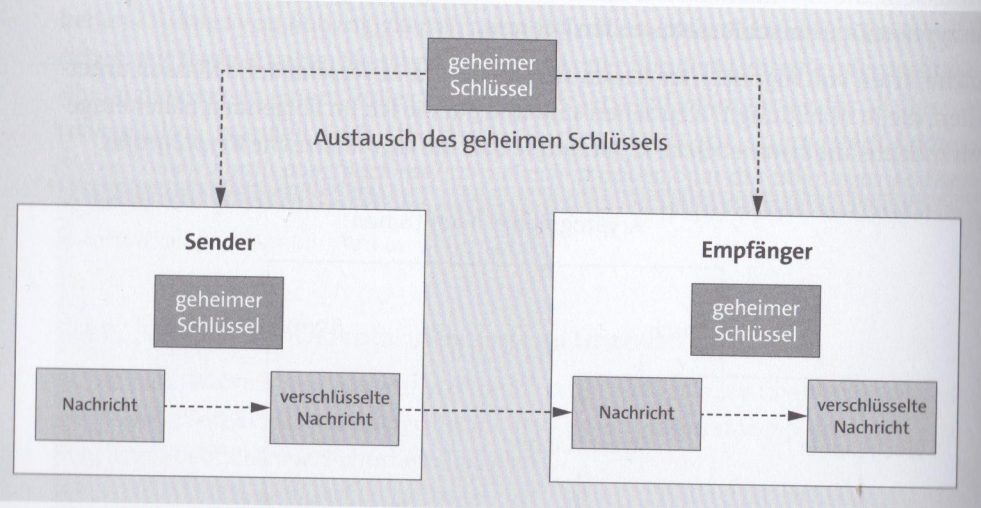


Abbildung 17.4 Das Prinzip der symmetrischen Kryptografie

Wegen der Verwendung nur eines Schlüssels sind Algorithmen zur symmetrischen Kryptografie in der Regel schneller als Algorithmen zur asymmetrischen Kryptografie, allerdings gibt es einen gravierenden Nachteil bzw. eine Schwachstelle: Bevor Sender und Empfänger mit der Verschlüsselung bzw. Entschlüsselung beginnen können, muss der geheime Schlüssel beiden zur Verfügung gestellt werden. Und da in der Regel der Sender der Daten derjenige ist, der den Schlüssel erstellt, muss also dieser Schlüssel irgendwie zum Empfänger gelangen, damit dieser die Daten entschlüsseln kann. Erfolgt die Übertragung dabei nicht sicher, kann der Schlüssel bei der Übertragung potentiell durch sogenannte *Man-in-the-Middle-Attacken* abgefangen werden, was ein enormes Sicherheitsrisiko für die anschließende – vermeintlich sichere, weil verschlüsselte – Kommunikation darstellt.

Aus diesem Grund kommt heutzutage häufiger die asymmetrische Kryptografie zum Einsatz.

17.2.2 Asymmetrische Kryptografie

Bei der *asymmetrischen Kryptografie* werden für die Verschlüsselung und für die Entschlüsselung der Daten **zwei unterschiedliche Schlüssel** verwendet, zum einen ein *privater Schlüssel* (Private Key), zum anderen ein *öffentlicher Schlüssel* (Public Key).

Wenn eine Nachricht mit dem öffentlichen Schlüssel verschlüsselt wird, kann sie nur mit dem privaten Schlüssel entschlüsselt werden (Abbildung 17.5), und wenn eine Nachricht mit dem privaten Schlüssel verschlüsselt wird, kann sie nur mit dem öffentlichen Schlüssel entschlüsselt werden.

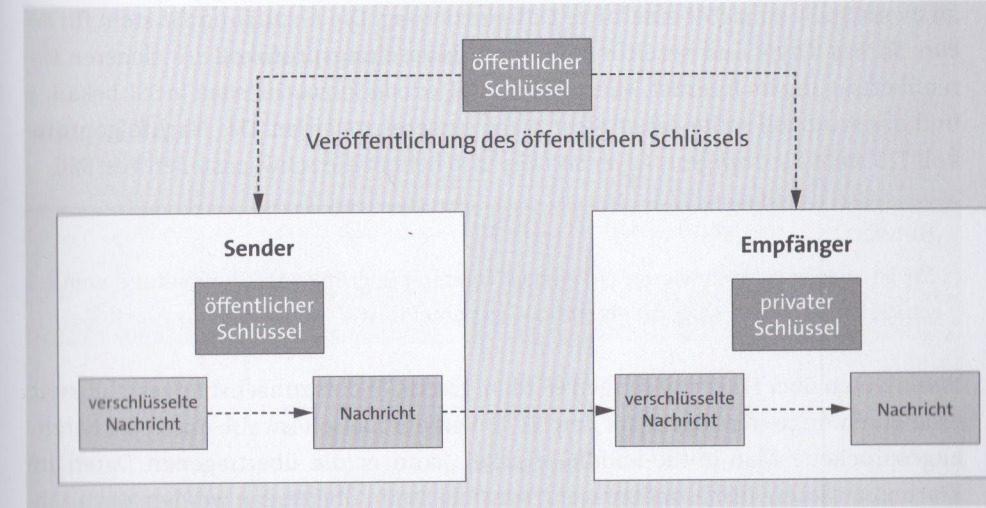


Abbildung 17.5 Das Prinzip der asymmetrischen Kryptografie

Auch wenn die Algorithmen zur asymmetrischen Kryptografie langsamer arbeiten als die zur symmetrischen Kryptografie, kommen in der Praxis erstere aufgrund der höheren Sicherheit häufiger zum Einsatz. So werden Algorithmen zur asymmetrischen Kryptografie in einer Vielzahl von Protokollen und Tools verwendet, beispielsweise:

- ▶ bei *SSL* (Secure Sockets Layer) und *TLS* (Transport Layer Security)
- ▶ bei der sicheren Variante von HTTP, dem Protokoll *HTTPS*, mit dessen Hilfe sich Daten sicher über das Web senden lassen
- ▶ bei dem Netzwerkprotokoll *SSH* (Secure Shell), mit dessen Hilfe man sich beispielsweise per Kommandozeile auf einem entfernten Webserver anmelden kann

- ▶ bei dem Programm *PGP (Pretty Good Privacy)*, das für das Signieren oder Verschlüsseln von E-Mails verwendet werden kann
- ▶ bei *DNSSEC (Domain Name System Security Extensions)*, das die Authentizität und Integrität von Daten sicherstellt, die im Domain Name System (DNS) übertragen werden

Für Sie am wichtigsten ist die Anwendung der Kryptografie im Hinblick auf SSL, TLS und HTTPS, weswegen ich auf diese Protokolle im Folgenden etwas näher eingehen möchte.

17.2.3 SSL, TLS und HTTPS

In der Praxis herrscht oft Unklarheit über die drei Begriffe *SSL*, *TLS* und *HTTPS*. Daher an dieser Stelle zunächst eine kurze Differenzierung: Die Abkürzung *SSL* steht für *Secure Sockets Layer* und bezeichnet ein **Verschlüsselungsprotokoll** zur sicheren Datenübertragung im Internet. Mittlerweile sind jedoch Schwachstellen in *SSL* bekannt, und das Protokoll sollte daher nicht mehr eingesetzt werden. Das **Nachfolgeprotokoll** *TLS* steht für *Transport Layer Security* und behebt die Schwachstellen von *SSL*.

Hinweis

SSL ist allerdings nach wie vor ein weit verbreiteter Begriff, sodass heutzutage, wenn von *SSL* gesprochen wird, oft eigentlich *TLS* gemeint ist.

Wenn Daten über *HTTP* übertragen werden, geschieht dies zunächst unverschlüsselt. Erlangt ein Angreifer Zugriff auf den *HTTP*-Verkehr (beispielsweise durch die bereits angesprochene *Man-in-the-Middle-Attacke*), kann er die übertragenen Daten im Klartext auslesen. Insbesondere wenn sensible Daten übertragen werden (dazu zählen beispielsweise Anmeldeinformationen wie Nutzernamen und Passwörter oder auch Kreditkarteninformationen), sollten Sie statt des »normalen« *HTTP* dessen sichere Variante *HTTPS (Hypertext Transfer Protocol Secure)* verwenden.

Syntaktisch ist *HTTPS* identisch mit *HTTP* (also beispielsweise bezüglich Headern etc.), die Daten werden für die Kommunikation allerdings unter Verwendung von *TLS* verschlüsselt. Dabei werden die Daten in beide Richtungen verschlüsselt übertragen, das heißt sowohl vom Client zum Server als auch vom Server zum Client.

Der Ablauf ist dabei folgender (Abbildung 17.6). Stellt der Client (bzw. Browser) eine Anfrage an den Webserver, schickt dieser zunächst seinen öffentlichen Schlüssel sowie ein *Zertifikat* an den Client. Der Client überprüft daraufhin zwei Dinge: zum einen die **Gültigkeit des Zertifikats** und zum anderen, ob es von einer **vertrauenswürdigen Zertifizierungsstelle** stammt.

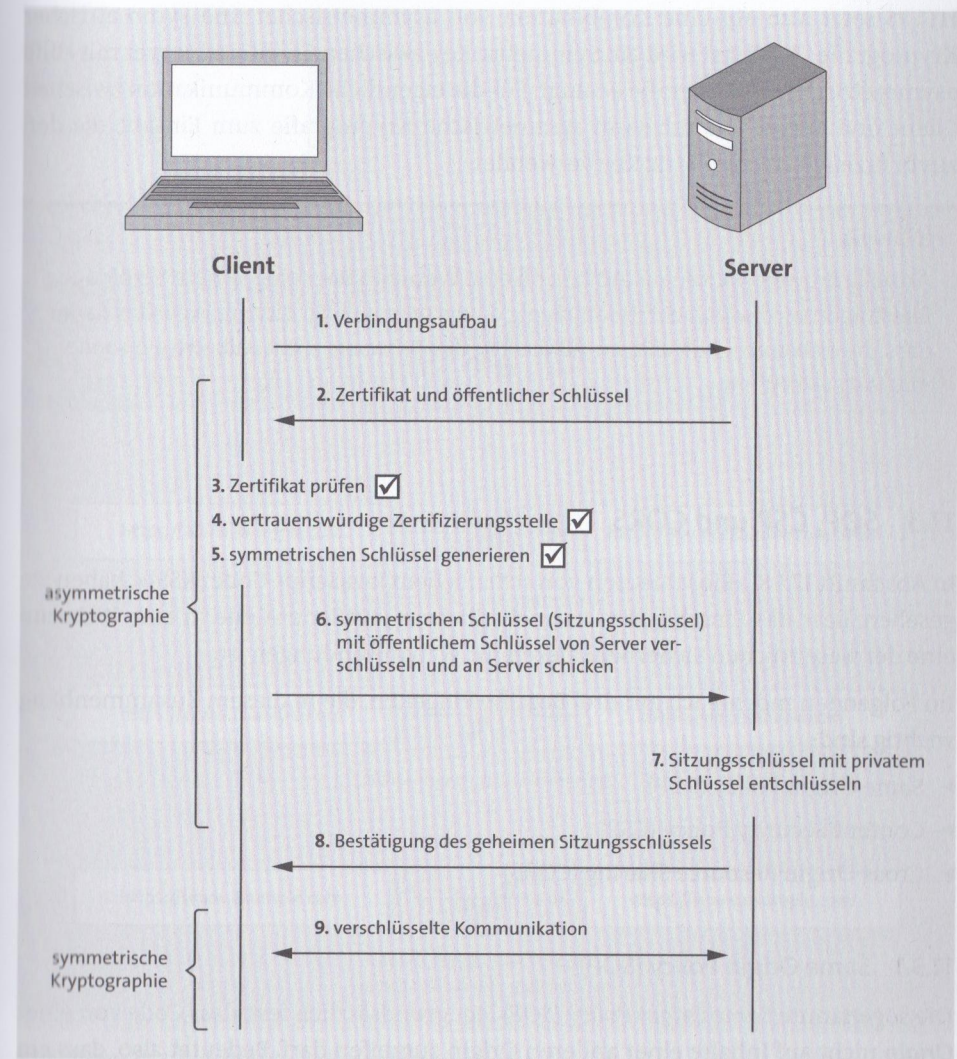


Abbildung 17.6 Das Prinzip von HTTPS

Nur wenn beides der Fall ist, erstellt der Client ebenfalls einen Schlüssel (den sogenannten *Symmetric Key*), verschlüsselt diesen mit dem öffentlichen Schlüssel und schickt ihn an den Webserver (**asymmetrische Kryptographie**). Dieser wiederum entschlüsselt den Symmetric Key mit seinem privaten Schlüssel (ebenfalls **asymmetrische Kryptographie**) und verschlüsselt im Folgenden mit dem Symmetric Key die an den Client zurückgesendeten Daten, die dieser wieder entschlüsseln kann (**symmetrische Kryptographie**).

HTTPS setzt also auf eine Kombination von asymmetrischer und symmetrischer Kryptografie: Zunächst wird ein Symmetric Key zwischen Client und Server mit Hilfe asymmetrischer Kryptografie erzeugt. Für die eigentliche Kommunikation zwischen Client und Server kommt dann symmetrische Kryptografie zum Einsatz, die den zuvor erzeugten Symmetric Key verwendet.

Hinweis

Aktuelle Browser weisen darauf hin, ob eine Webseite über eine sichere Verbindung übertragen wird oder nicht. Unabhängig davon erkennen Sie sichere Websites daran, dass die entsprechende URL im Adressfeld des Browsers nicht mit »http«, sondern mit »https« beginnt.

17.3 SOP, CSP und CORS

In Abschnitt 17.1.8, »Einschleusen von schädlichem JavaScript-Code (XSS)«, haben Sie gesehen, dass das Einschleusen von schädlichem JavaScript-Code in eine Webseite eine der wesentlichen Sicherheitslücken für Webanwendungen ist.

Im Folgenden möchte ich auf drei Begriffe eingehen, die in diesem Zusammenhang wichtig sind:

- ▶ Same Origin Policy (SOP)
- ▶ Content Security Policy (CSP)
- ▶ Cross-Origin Resource Sharing (CORS)

17.3.1 Same Origin Policy (SOP)

Die sogenannte *Same Origin Policy* (SOP) legt grundsätzlich fest, dass Code von einer *Origin* nicht auf Inhalte einer anderen Origin zugreifen darf. Bedeutet also, dass ein im Browser ausgeführtes JavaScript per HTTP nur auf Ressourcen derselben Domain zugreifen kann, das dynamische Laden von Ressourcen von einer anderen Domain – das Ausführen sogenannter *Cross-Origin Requests* – jedoch unterbunden wird.

Genauer gesagt muss die angefragte Ressource über das gleiche **Protokoll**, den gleichen **Host** und den gleichen **Port** wie der JavaScript-Code erreichbar sein. Wenn Sie beispielsweise von einer Webseite `https://www.example.com` per JavaScript eine JSON-Datei einlesen möchten und diese etwa unter Verwendung der Fetch API laden möchten, muss die JSON-Datei ebenfalls auf `https://www.example.com` liegen (Abbildung 17.7).

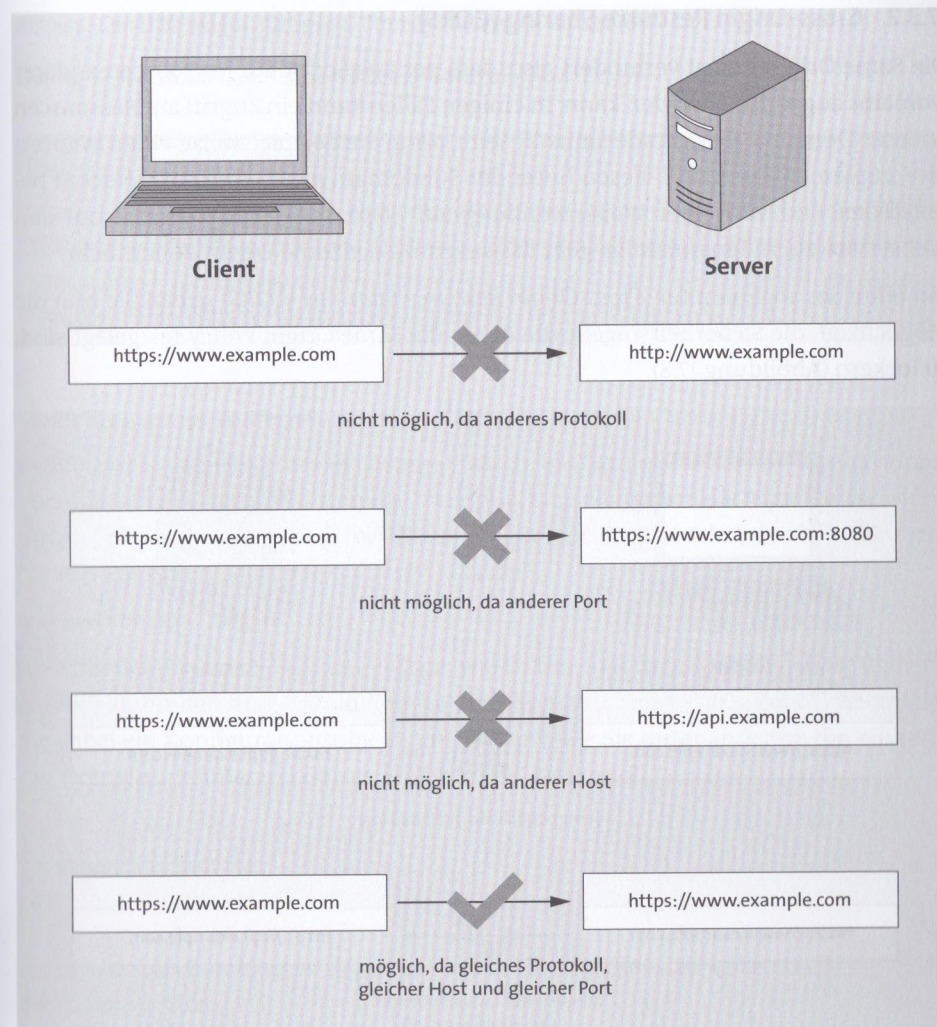


Abbildung 17.7 Das Prinzip der Same Origin Policy

Hinweis

Die Same Origin Policy betrifft nur das **dynamische Laden von Ressourcen** aus JavaScript-Code heraus. Ressourcen, die Sie direkt über HTML einbinden, können dagegen von anderen Domains geladen werden. Dazu zählen:

- ▶ JavaScript-Dateien, die über das `<script>`-Element eingebunden werden,
- ▶ CSS-Dateien, die über das `<link>`-Element eingebunden werden,
- ▶ Bild-, Audio-, Videodateien etc., die über die Elemente `<img>`, `<audio>`, `<video>`, `<object>` und `<applet>` eingebunden werden, sowie